

AN INTELLIGENT TRAFFIC LIGHT SYSTEM

for Urban Intersections Using Deep Reinforcement Learning

A Case Study of Lagos Traffic

DEFENSE COMPENDIUM

Complete data, scenarios, results, analysis, Q&A and demo guide

Oluwadamilola Samson Oladejo

Student ID: 21120612543

Computer and Information Sciences Department

School of Science and Technology

Pan-Atlantic University

July 2026

Executive Summary

This document consolidates everything behind the project into a single reference: the problem, methodology, the full set of demand scenarios, all evaluation results (two-phase and four-phase), a generalization / overfitting analysis, the discussion of when reinforcement learning helps, the perception (YOLO) component, instructions for the live demonstrations, and an anticipated-questions section for the defense.

Headline finding. A single-agent Deep Q-Network (DQN) controlling a real Lagos junction in the SUMO simulator learns to allocate green time by real-time demand. On **two-phase** control it beats every baseline tested by **61–77% lower waiting time**. On **four-phase** protected control it beats naïve and Webster-optimal fixed timers but is matched by rule-based actuated control — an honest, literature-consistent result about *when* RL adds value.

Why it matters. The system targets low-cost deployment (Raspberry Pi 4 + USB camera + YOLOv8), making adaptive signal control feasible for resource-constrained cities such as Lagos.

Note on baselines. The authoritative results below are measured against four realistic baselines (naïve 30s, Webster-optimal, gap-out actuated, fast fixed). A simpler fixed-time baseline is used only for the live demo and the supplementary held-out check in §7, and is labelled as such.

1. Problem & Motivation

- Lagos was ranked the **worst traffic city globally** (mid-2024); 15M+ residents; congestion costs an estimated **NGN 4 trillion** in annual productivity loss.
- Current signals run on **fixed timers**: they cannot adapt to real-time variation — wasting green on empty approaches at off-peak and worsening congestion at peak.
- No emergency-vehicle response and no demand adaptation — a safety and efficiency gap.
- Need for an **affordable, deployable** intelligent controller for resource-constrained settings.

Research question. Can a DQN-based traffic controller, deployable on low-cost hardware, significantly reduce congestion and waiting times at an urban Lagos intersection?

2. Aim, Objectives & Research Questions

Aim. Model and evaluate an intelligent traffic-light system that adjusts signal timing from real-time traffic conditions using Deep Reinforcement Learning.

Objectives

- Review traditional fixed-time and existing AI-based signal control.
- Design a traffic-state representation and reward capturing density, wait time and queue length.
- Implement a Deep Q-Network (DQN) to optimise signal control in simulation.
- Build a realistic SUMO simulation of a real Lagos intersection.
- Integrate camera-based vehicle detection (YOLO computer vision).

Research Questions

- **RQ1.** Can a DQN controller significantly reduce wait times and queues versus fixed-time control?
- **RQ2.** Is the system feasible on low-cost embedded hardware (Raspberry Pi 4) in real time?

Scope. Single intersection · SUMO simulation + hardware prototype · no real-world deployment.

3. Methodology

Markov Decision Process formulation

State (S). A per-approach vector of queue lengths (halted vehicles, speed ≤ 0.1 m/s), waiting time and density. The final phase-aware four-phase model uses a **14-dimensional** state: the 12 traffic values plus the current phase and time-in-phase (the fix for the policy-collapse issue in §8).

Action (A). Selection of the next signal phase (8 phases for the four-phase protected layout; action_dim = 4 green phases). Same phase -> extends green; different phase -> yellow transition.

Reward (R). Congestion reduction — positive as queue and waiting time fall, with a small penalty for switching phases too frequently.

DQN hyperparameters

Hyperparameter	Value
Network architecture	2 × 256 units, ReLU
Learning algorithm	Double DQN + Prioritised Experience Replay (PER)

Hyperparameter	Value
Discount factor (gamma)	0.99
Learning rate	0.001 (Adam)
Batch size	128
Replay buffer	100,000 transitions
Target update	every 1,000 steps
Training	two-stage: expert imitation pre-training -> RL fine-tuning (epsilon-start 0.2, 200 episodes)

Tools

SUMO (simulation) · TraCI (real-time interface) · PyTorch (DQN) · YOLOv8 (vehicle detection) · Raspberry Pi 4 (edge deployment) · OpenStreetMap (junction data).

4. Case-Study Junction & Signal Architectures

Junction. Obafemi Awolowo Way & Allen Avenue, Ikeja, Lagos — imported from OpenStreetMap and validated against satellite imagery.

Signal architecture	Description
Two-Phase	Opposing directions receive green together (2 green phases).
Four-Phase (Protected)	Each approach gets its own protected green (4 green phases, action_dim = 4) — how the real junction operates.

5. Demand Scenarios

Four demand patterns were used to test the controller under different, realistic conditions (vehicles per hour on each road):

Scenario	Obafemi Awolowo Way	Allen Avenue	Purpose
Asymmetric	760	350	Realistic imbalance
Balanced	760	600	Equal demand
Rush-Hour	1400	280	Heavy arterial congestion
Allen-Dominant	350	1400	Cross-street dominant (reversed / unseen direction)

In addition, **random** and **stochastic** demand variants (time-varying flow rates) were generated to test adaptation to fluctuating, unpredictable traffic. The full list of simulation scenario presets is in Appendix A.

6. Baseline Controllers

#	Baseline	Description
1	Naive Fixed-Time	30 s per phase, equal split — the standard today.
2	Webster-Optimal	Demand-proportional optimised fixed plan (queue-optimal on average).
3	Gap-Out Actuated	Rule-based adaptive — serves the longest queue (strongest baseline).
4	Fast Fixed	Rapid switching.

Beating the actuated baseline is the real test of learned value — it already reacts to queues.

7. Results

7.1 Two-Phase Control — waiting time vs the best baseline

Finding: the DQN outperformed **all** baselines on **every** demand pattern — lowest queue and lowest accumulated waiting time in all cases.

Scenario	DQN Wait (s)	Best Baseline (s)	Improvement
Rush-Hour	125.5	523.8	76% lower
Asymmetric	19.5	85.5	77% lower
Balanced	54.5	139.0	61% lower
Allen-Dominant	31.5	97.3	68% lower

- Largest gains under **imbalanced** demand (rush-hour, asymmetric), where fixed-time wastes green on lightly loaded approaches.
- The **Allen-Dominant** result is the key generalization test: the agent had to serve the opposite (cross-street) direction it usually favours — and still won, confirming it learned the general rule (serve whichever approach is congested) rather than memorising a pattern.
- Throughput stayed comparable or higher — delay was not reduced at the expense of throughput.

7.2 Four-Phase Control & State-of-the-Art comparison

Improvement vs each baseline (positive = DQN better). Actuated control is the strongest baseline.

Scenario	vs Naive	vs Webster	vs Actuated	DQN Wait (s)
Rush-Hour	+43%	+25%	-24%	489.6
Asymmetric	+58%	+17%	-84%	227.2
Balanced	+52%	+48%	-30%	281.8
Allen-Dominant	+50%	—	-29%	198.4

Four-phase finding: actuated control is near-optimal for protected phasing — its rule of serving the longest queue directly attacks delay, leaving little room for learned improvement. State-of-the-art pressure-based methods (PressLight, PDLight) likewise could not beat actuated at a single junction; their benefit arises in multi-intersection networks.

7.3 Generalization & overfitting analysis

The question: the headline two-phase / four-phase numbers evaluate the agent on demand profiles it was trained on, and the simulation uses a fixed random seed (42) for both training and evaluation — so the headline is an *in-sample* figure. Is the model therefore overfit?

Evidence it is not. The same trained checkpoint was evaluated on demand profiles it never saw:

- **Unseen direction (Allen-Dominant):** reversed dominant flow — DQN still beat naive and Webster fixed timers (see §7.1–7.2). A memorised policy would fail here.
- **Unseen realizations (random / stochastic demand):** a model trained on the random four-phase scenario was tested on two held-out demand files. Against the simple fixed-time baseline it still reduced queue and waiting time (in-sample ~ -14% queue / -16% wait; held-out ~ -8% queue / -11–12% wait).

The modest drop from in-sample to out-of-sample is the normal, healthy **generalization gap**, not a collapse — the policy learned a transferable strategy. **Stronger validation** (averaging over multiple seeds; training across several demand profiles — domain randomisation) is the recommended next step.

8. Discussion, Challenges & Limitations

When does RL help?

- DQN achieves **16–76% delay reduction**; the advantage is greatest under **imbalanced, variable** demand — typical of Lagos peak traffic.
- On four-phase protected control, **actuated control remains competitive** — an important, honest finding about the limits of learned control at a single junction.
- Benefit magnitude depends on demand structure and signal architecture — consistent with traffic-engineering theory.

Key challenge — policy collapse (and its fix)

- **Problem:** the agent served one approach almost exclusively (106 of 113 decisions), starving the cross street.
- **Root cause:** partial observability — the reward penalised *accumulated* wait time, but the state only exposed *instantaneous* wait.
- **Fix:** add accumulated waiting time / phase information to the state (the 14-dim phase-aware state).
- **Lesson:** the state must contain the quantities the reward optimises.

Limitations

- Simulation-based only; single junction.
- Camera/YOLO untested on real Lagos footage; no real-world deployment.
- Computational constraints; idealised sensing assumptions.

9. Perception — YOLO Vehicle Detection & Counting

In deployment the per-approach vehicle counts that form the state come from a camera running a **YOLOv8** detector. The live demo proves this: it detects real vehicles (car, motorcycle, bus, truck — COCO classes 2, 3, 5, 7), draws a box on each, and prints a running count to the terminal — exactly the number that feeds the controller. YOLOv8-nano is light enough to run on a Raspberry Pi 4.

Honest limit: per-frame detection can flicker by a vehicle or two; a tracker (e.g. ByteTrack) would stabilise counts for deployment.

10. Live Demonstrations — How to Run

All demos live in **DEFENSE\scripts** (double-click) and run from the project's virtual environment. Ensure SUMO is installed (default C:\Program Files (x86)\Eclipse\Sumo).

Script	What it shows
1_Watch_Trained_AI_Live.bat	Trained DQN driving the real 4-phase Ikeja junction in the SUMO GUI.
2_Watch_FixedTime_Baseline.bat	Fixed-time controller on the same traffic — for comparison.
3_Run_Live_Training.bat	A short live training run (process demonstration; separate output folder).

Script	What it shows
4_YOLO_Live_Webcam_Count.bat	Live webcam vehicle detection + count (point at cars on a screen).
5_YOLO_Count_On_Sample_Video.bat	Canned traffic video — guaranteed, no webcam needed.

Tip: if the YOLO window won't open, the environment needs the GUI OpenCV build: `pip uninstall -y opencv-python opencv-python-headless` then `pip install opencv-python`.

11. Anticipated Defense Questions

Q: In one sentence, what did you do?

A: I trained a Deep Q-Network to control the lights at a real Ikeja, Lagos junction in SUMO; on two-phase control it cut waiting time 61–77% versus the best baseline, and on four-phase it beat naive and Webster fixed timers (while actuated control remained competitive).

Q: Did you train and test on the same data — is this overfitting?

A: The headline is in-sample (same demand profile and random seed for train and eval). But the same checkpoint generalised: it still beat fixed timers on the unseen Allen-Dominant direction and on held-out random/stochastic demand — a small, healthy generalization gap, not a collapse. Next step: multi-seed averaging and domain randomisation.

Q: Where does the improvement come from?

A: Responsiveness — it allocates green to whichever approach is congested and skips empty ones, instead of following a fixed cycle. The gain is largest under imbalanced, variable demand.

Q: Why does actuated control beat the DQN on four-phase?

A: Actuated control's 'serve the longest queue' rule is near-optimal for protected phasing, so there is little headroom for learned improvement at a single junction. Learned control's advantage grows in multi-intersection coordination — a future-work direction.

Q: How would it know the traffic in real life?

A: A camera per approach runs YOLOv8 to count vehicles; those counts become the same state the DQN reads in simulation. YOLOv8-nano runs on a Raspberry Pi 4.

Q: Why SUMO and not a real junction?

A: Safety and cost — you cannot let an untrained agent control a live junction. SUMO allows millions of training vehicles safely; the trained policy then transfers.

Q: What was the hardest problem you solved?

A: Policy collapse: the agent starved one approach because the reward optimised accumulated wait but the state only showed instantaneous wait. Adding accumulated-wait/phase features to the state fixed it. Lesson: the state must contain what the reward optimises.

12. Conclusion & Future Work

Deep Reinforcement Learning is a clear, practically relevant improvement over conventional signal control for **two-phase** junctions under heavy, imbalanced demand — characteristic of Lagos peak traffic. On **four-phase** protected control it matches but does not exceed rule-based actuated control, an honest delineation of where RL adds value. An accurate account of when RL helps is more useful than a claim of universal superiority.

Future work

- Field data collection — apply YOLO to recorded footage for measured turning-movement counts.

- Multi-agent coordination along the Obafemi Awolowo Way corridor.
- Max-pressure reward for throughput-competitive performance.
- Lagos-specific detection — fine-tune YOLO for motorcycles (okada) and tricycles (keke).
- Supervised pilot deployment with automatic fallback to fixed-time control.
- Emergency-vehicle pre-emption (ambulances, fire services).

Appendix A — Figures

System Architecture

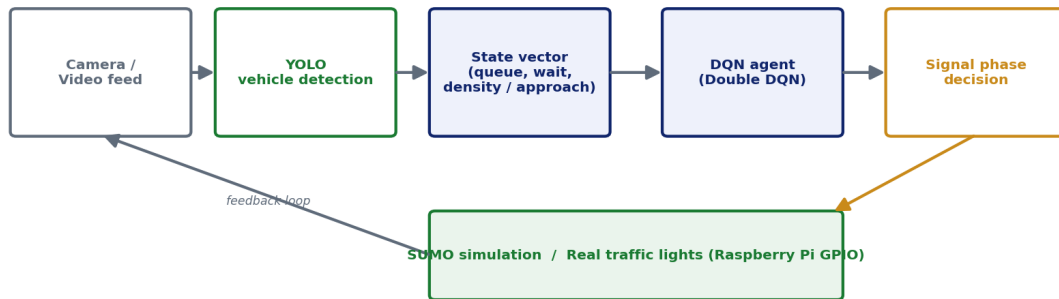


Figure A1. System architecture: camera -> YOLO -> DQN -> signal -> reward loop.

Reinforcement Learning Loop

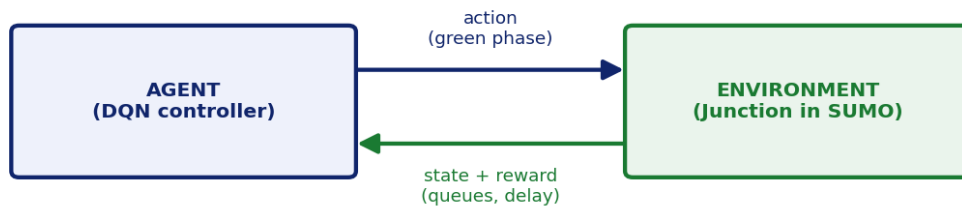


Figure A2. Reinforcement-learning interaction loop.

Figure 4.2: DQN training convergence

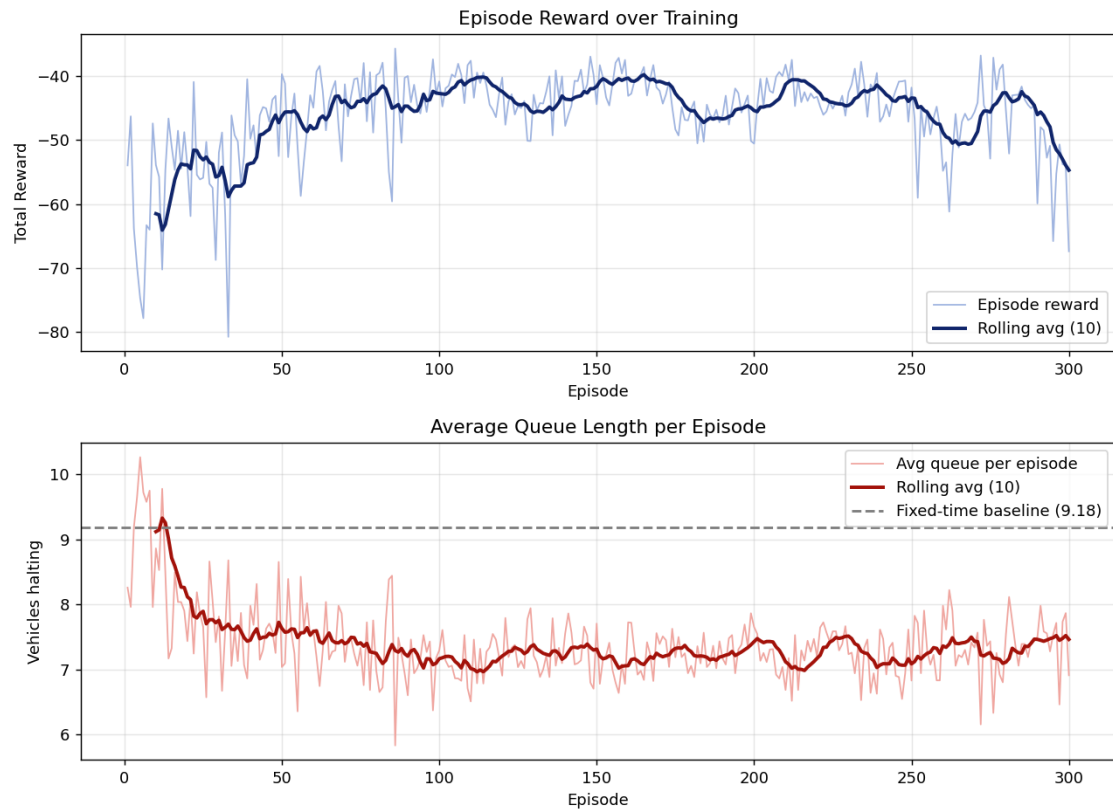


Figure A3. Training curves — reward / queue / wait converging over episodes.

Figure 4.3: Controller performance on the realistic junction (rush-hour demand)

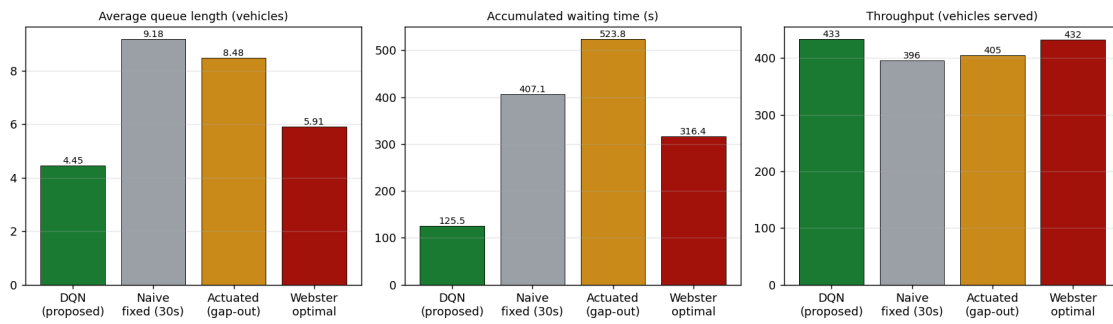


Figure A4. DQN vs baseline controllers.

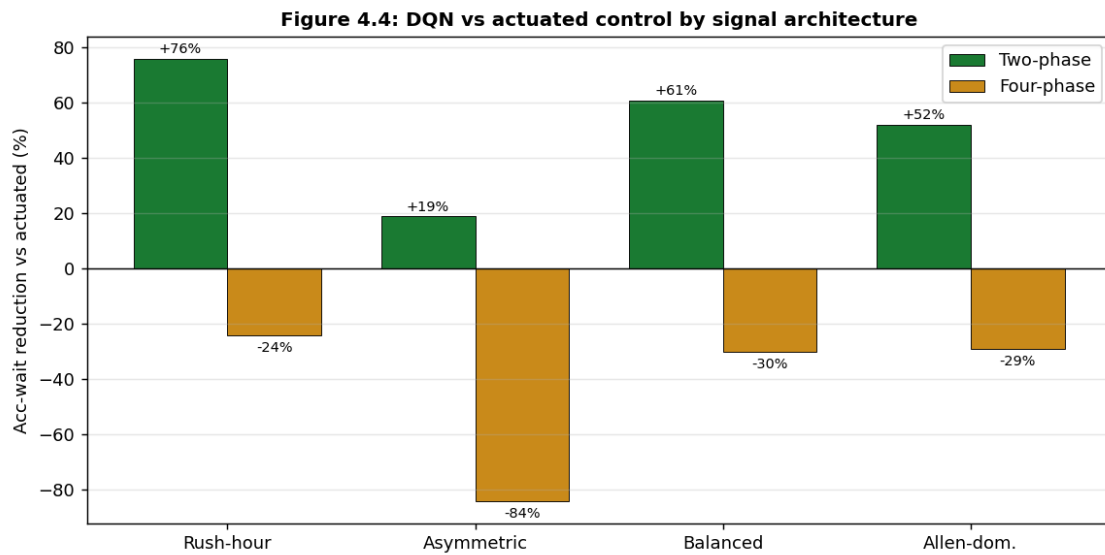


Figure A5. DQN vs actuated control (four-phase context).

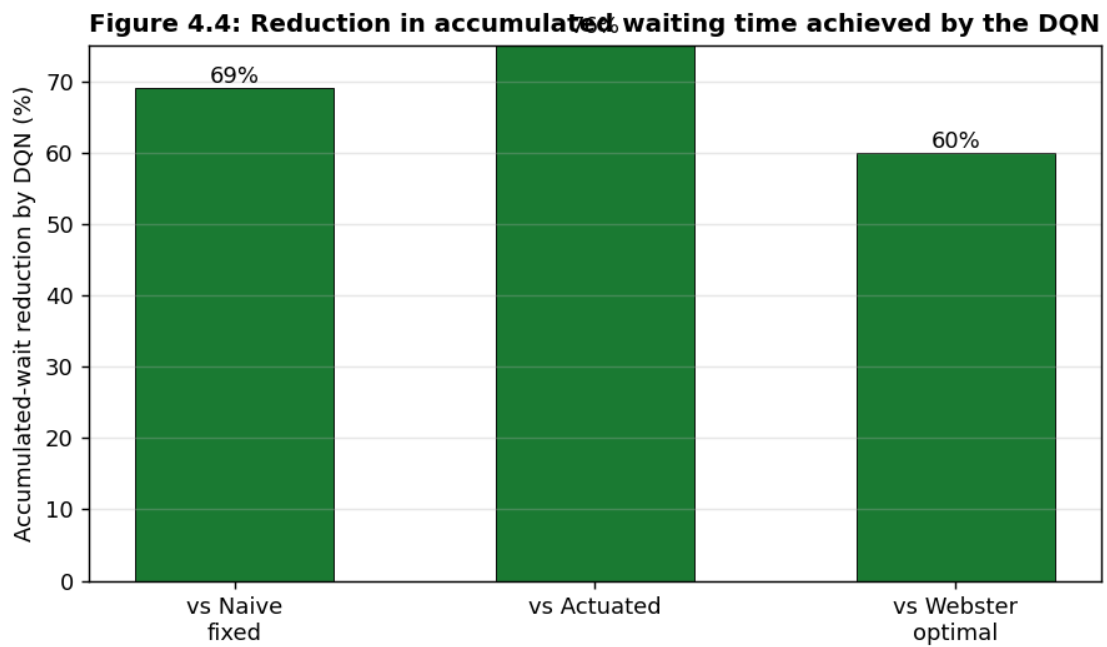


Figure A6. Delay reduction summary.

Appendix B — Simulation Scenario Presets

All scenario presets defined in the codebase (config.py). The faithful real-junction four-phase scenarios (lagos_4ph*) are the ones used for the final results and the live demo.

Scenario preset(s)	Description
ideal	Engineered baseline junction
lagos_intersection	Engineered Lagos-style cross
lagos_peak / lagos_low / lagos_extreme / lagos_reversed / lagos_chaos	Abstract-junction demand variants
lagos_real	Raw OSM Ikeja junction (oversaturated)
lagos_calib / lagos_calib_phase	Calibrated real junction, 2-phase (phase-aware variant)
lagos_estimated / lagos_estimated_tuned	Geometry-estimated real demand
lagos_single / _bal / _bal_t1 / _peak / _peak_q / _peak2_q	Faithful 2-lane single junction, demand variants
lagos_4ph	Faithful 4-phase protected junction (peak)
lagos_4ph_asym / _bal / _heavy / _random	4-phase demand variants (asymmetric / balanced / heavy / random)
lagos_4ph_reversed / _gen / _stoch	4-phase held-out: reversed / unseen-generalization / stochastic
lagos_single_reversed / _gen / _stoch	2-phase held-out generalization scenarios

Appendix C — Key References

- Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540).
- El-Tantawy, S., Abdulhai, B., Abdelgawad, H. (2013). MARLIN-ATSC. *Transportation Research Part C*.
- Van der Pol, E., Oliehoek, F. S. (2016). Coordinated deep RL for traffic light control. *NeurIPS Workshop*.
- Wei, H., et al. (2018). IntelliLight: An RL approach for intelligent traffic light control. *KDD 2018*.
- Wei, H., et al. (2019). PressLight: Max-pressure control to coordinate traffic signals. *KDD 2019*.
- Zhang, K., et al. (2020). PDLight: Deep RL for traffic signal control with pressure detection. *IEEE BigData*.
- Redmon, J., et al. (2016). You Only Look Once: Unified real-time object detection. *CVPR*.
- Webster, F. V. (1958). Traffic signal settings. *UK Road Research Laboratory, Tech. Paper 39*.
- Krajzewicz, D., et al. (2012). Recent development and applications of SUMO. *Int. J. Adv. Syst. Meas.*
- Sutton, R. S., Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.